Angel Almanzar
Professor Holden-Gouveia
Linux Administration
3/**24**/26

AwkLab

The goal of this **fourth** assignment Awklab was to be as clear as possible. This assignment does not have anything color coded, and is instead replaced with **bold** lettering or underlined words for each question. Along with the question number there is the **name of the question** it is referring to, to make it easier to **read** and find. For example "Printing all the first names in a file" etc.

For each question I did the best I could to answer the question, reading documentation, going to tutoring for clarification or tweaking commands, and piecing things together to create a command or a similar command to an example like grymoire or other sources. Below each question answered are the sources I used. **This documentation aimed to be as clear and straightforward as possible**, also easy for the reader to understand each command step by step.

Question **1.**

Cat AwkLab.data is a command that shows everything within the file.

Printing all the first names in the file AwkLab.data.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk '{print $1}' AwkLab.data
Samuel
Ponder
Angua
Susan
Tiffany
Sacharissa
Adora
Frodo
Tom
Peregrin
Samwise
A.A.
Antoine
Adalgrim
Bandobras
Belladonna
Eglantine
Mirabella
Ferumbras
Gerontius
aavbox26@Ubuntu1:/mnt/fileshare$
```

The following command prints **all the first names** of the awk file named **AwkLab.data.** The command written in the example above was as follows....

awk '{print $1}' AwkLab.data. Make sure to include a space **after awk** and after the second **single quote**, to prevent any errors or commands not found etc.

Command you would write....

awk '{print $1}' Filename. Make sure to include a space after awk and after the second **single quote ('),** to prevent any errors or commands not found etc.

This command has what I like to call **four** parts, when broken up step by step. To begin the command starts with **awk, awk** stands for **a-Aho, w-Weinberger, k-Kernighan.** These are the last names of the people who created the program. **Awk** is used for text processing, analyzing, and filtering different data.

For this command, **print** outputs what is in the file **AwkLab.data.** The dollar sign one **($1), $ dollar sign** means to reference a field in **awk**, **the one means to reference the first field** and adding any other number like **$2** or **$3** and so on **means to reference that many number of fields.** What is a **field** in awk? A field is small pieces of data within a line of text, separated by whitespace like words. This command begins with a **single quote** and ends with a **single quote** with a **curly bracket** in front of both single quotes, after this the **filename** is written.

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-1 awk **tutorials.**

https://www.grymoire.com/Unix/AwkRef.html awk **references.**

https://www.grymoire.com/Unix/Awk.html#uh-17 Number of fields.

https://www.gnu.org/software/gawk/manual/html_node/Fields.html Important **syntax.**

https://www.geeksforgeeks.org/linux-unix/awk-command-unixlinux-examples/ What is **awk**?

https://askubuntu.com/questions/524568/what-is-the-full-form-of-awk **Full name for awk,** found this interesting.

https://www.youtube.com/watch?v=4HpWO_RAMq4&t=152s Watched the video for **examples** and better understanding of **awk.**

Question **2.**

Printing phone numbers after the names Tom and Frodo.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '/Tom|Frodo/ {print $1, $2}' AwkLab.data
Frodo Baggins (206) 548-1278
Tom Bombadil (916) 348-4278
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

The next command is printing phone numbers for **Tom** and **Frodo** after their names. The command written in the screenshot above was as follows,

Awk -F: '/Tom|Frodo/ {print $1 , $2}' AwkLab.data.

For this command we start with **awk** once again, as this is the program we are using. **-F:,** Stands for **Field Separator, Colon (:).** We then write a **single quote** a **forward slash (/),** the pattern/sequence which is **Tom,** then **pipe (|),** followed by another pattern **Frodo, forward slash (/), curly brackets with print, $ dollar sign 1**, **$ dollar sign 2**, then to end the command we then write the filename.

The source for the data is from the **AwkLab.data** file. For this command, the forward slashes are used to find the pattern, and the **pipe** is used to separate the search and to look for **both Tom** and **Frodo** pattern/sequence. The comma in between the **$1 and $3 fields** tells **awk** to read a space in between the name and the data. The first field looks for the **first field in the file, which is the full name.** The second field looks for the phone numbers within the file.

File separators in awk are, **FS= ":" ; , -F.**

Field Data when using -F:

$1 is for the names, **$2** is for the phone numbers, **$3** is for donation amount **1**, **$4** is donation amount **2, $5** is donation amount **3.** All fields are separated by colons, each field can also be called columns. **Column 1-5** for this file named **AwkLab.data.**

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-1 **awk basics and printing fields.**

https://www.grymoire.com/Unix/Awk.html#uh-17 Number of fields.

https://www.grymoire.com/Unix/Awk.html#uh-53 **awk Patterns.**

https://www.grymoire.com/Unix/Regular.html#uh-10 Back references.

https://www.grymoire.com/Unix/Regular.html#uh-12 Extended Regular Expressions.

https://www.gnu.org/software/gawk/manual/gawk.html#Regexp-Patterns Regular Expressions as Patterns.

Tutoring Assistance.

Question **3.**

Printing Peregrin's full name and phone number with the area code only.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk '/Peregrin/ {print $1, $2 substr ($3 , 0, 0)}' AwkLab.data
Peregrin Took:(510)
aavbox26@Ubuntu1:/mnt/fileshare$
```

This command aimed to get the full name of the person named **"Peregrin"**, along with the area **code** of their phone number associated with them. So what exactly does this command do step by step? Awk is the program. First is the **single quote (')**,

followed by the **pattern search** forward slashes **(//)**, within the slashes is the pattern, the pattern we are searching for which is Peregrin. Following the pattern we write the **curly brackets { }** then we want to print the first and second fields which are in this command **($1, $2) ,** followed by the **substr (substring)** function. The substring function is used to extract a part of a string. Then we write the **parentheses ()**, within the parentheses we write the $3 field, following is **0, 0**.

Zero (**0)** is only getting the beginning, when using **0 , 1** you get the first number outside of the parentheses, if you put **0 , 2** you will get the second number for the phone number and writing other sequences gives you the entire phone number and donation amounts. To get the entire amount it would be **0, 8.**

The full command written was,

awk '/Peregrin/ {print $1, $2, substr($3, 0, 0)}' AwkLab.data.

What you would write is,

awk '/pattern/ {print $1, $2, substr($3, 0, 0)}' Filename.

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-1 awk basics and printing fields.

https://www.grymoire.com/Unix/Awk.html#uh-43 The Substring function.

https://www.grymoire.com/Unix/Awk.html#uh-53 awk patterns.

https://www.gnu.org/software/gawk/manual/gawk.html#String-Functions awk guides.

https://www.geeksforgeeks.org/linux-unix/built-functions-awk/ awk functions.

https://stackoverflow.com/questions/77159814/what-are-delimiters-for-in-regular-expressions#:~:text=Some%20languages%20have%20literals%20for,this%20is%20the%20closing%20delimiter. **What are pattern delimiters and delimeters in regular expressions?**

Question **4.**

Printing all phone numbers with the area code **(123) with names.**

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '/123/{print $1, $2}' AwkLab.data
Antoine de Saint-Exupery (123) 978-6432
Belladonna Took (123) 978-5754
Eglantine Took (123) 978-3574
aavbox26@Ubuntu1:/mnt/fileshare$
```

Colons (:) are used as separators in the file, they separate the names, phone numbers, and donation amounts.

How would we write a command that prints all phone numbers with an **area code** that starts with **123** and the names associated with them? To do this we would write this command **awk -F: '/123/{print $1, $2}' AwkLab.data .** Looking at the data in the file named AwkLab.data in Microsoft Word I saw that there were only three different instances where the area code in a person's phone number began with **123.**

The three different people whose phone number began with 123 were **Antoine, Belladonna, and Eglantine.** Breaking down the command we can see that, the command begins with awk, **-F:,** Stands for **Field Separator, Colon (:).** Then a single quote, the slashes pattern search containing the 123 pattern/sequence. The reason **123** is written within the slashes is because we want to look specifically for the phone numbers that start with **123.**

We then write a curly bracket followed by print and the field we want to print for is **the first $1 and the second $2 fields.** When writing **/123/** we look for **123** next to each other, the pattern/sequence. When writing the field **$1** we are looking for the name and the second field is searching for the phone numbers. **123** is the area code for the phone number for the three different people.

We then close this command with another curly bracket and write a single quote as we began, the command with one then we write the filename. The filename in this example is AwkLab.data.

Sources:

https://www.gnu.org/software/gawk/manual/gawk.html#Expression-Patterns This source shows Expressions as Patterns, with examples.

https://www.gnu.org/software/gawk/manual/gawk.html#Regexp-Patterns Regular Expressions as Patterns.

https://community.unix.com/t/printing-names-using-awk/309256 Printing names.

https://www.youtube.com/watch?v=4HpWO_RAMq4&t=152s Used **examples** from video.

Question **5.**

Printing all last names beginning with either T or D (excluding middle names).

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '{sub (/^.* /,"",$1)} $1~/^[TD]/ {print $1}' AwkLab.data
Dearheart
Took
Took
Took
Took
Took
Took
Took
Took
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

-F: is the **field separator.** Single quote **(‘)** is the beginning of the command, we then write a curly bracket, the sub command, parentheses, then we start the search pattern using **Regex** with a **slash (/), ^ caret**, caret signifies the start of a line. We then write a **period (.)**, which means to match any character, the **asterisk (*)** means anything can come after the period **(.) 0 or more times**. After the asterisk it is important to add a single space, the space looks for a space in the text line, we do this because the file is written as first name, space middle name, space last name.

This is how we get to the first name, middle name, last name and stop at the last name because the last name does not have any spaces in them. The **comma (,)** is used to separate sections of the sub command, the two **double quotes (“”)** represent our replacement, which is replacing something with nothing, acting as a way to delete the

first and middle names from our output. The **tilde (~)** tells awk to search **$1** matching an uppercase **T or D**, we then print **$1** to print our result.

The command written in this example was….

Awk -F: '{sub (/^.* /,"",$1)} $1 ~/^[TD]/ {print $1}' AwkLab.data

<u>Field Data when using -F:</u>

$1 is for the names, **$2** is for the phone numbers, **$3** is for donation amount **1**, **$4** is donation amount **2, $5 is donation amount 3.** All fields are separated by colons, each field can also be called columns. **Column 1-5.**

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 Awk Tutorials.

https://www.grymoire.com/Unix/Regular.html#google_vignette Regular Expressions.

https://www.gnu.org/software/gawk/manual/html_node/Regexp-Usage.html Pattern Matching.

https://www.gnu.org/software/gawk/manual/html_node/String-Functions.html Sub Command.

https://www.gnu.org/software/gawk/manual/html_node/Bracket-Expressions.html Bracket Examples.

Tutoring assistance.

Question **6.**

<u>Printing all first names **containing 4 characters or less.**</u>

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk 'length($1) <= 4 {print $1}' AwkLab.data
Tom
A.A.
aavbox26@Ubuntu1:/mnt/fileshare$
```

We start the command with a **single quote (‘)** we then write the **length** command, followed by **parentheses field 1 ($1).** We want to match only the names that contain four characters or less so we write greater than or equal to 4 **(< greater than, = equals, four 4)** we then want to print the first field which is the name with a **curly bracket {}** then we write the filename.

The command written in the example was..

awk ‘length ($1) <= 4 {print $1}’ AwkLab.data.

The command you should write for your file should be…

awk ‘length ($1) <= 4 {print $1}’ Filename.

Sources:

https://medium.com/@linuxrootroom/understanding-the-awk-length-function-ac6b360dd387 **Filtering data example.**

Question **7.**

Printing the first names and area codes of all those in the **(916) area code.**

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk '/916/ {print $1, "(916)"}' AwkLab.data
Sacharissa (916)
Tom (916)
A.A. (916)
aavbox26@Ubuntu1:/mnt/fileshare$
```

The following command prints the names of people who are associated with the **916 area code**. We start the command with a **single quote (‘)** we then look for the pattern/sequence which in this case would be **916.** The slashes **(/ /)** search for the 916 area code. We then write a **curly bracket {}** we want to print the first field which would be the names, then we want to print 916 area code, we then write the filename. The names associated with the **916** area code are **Sacharissa, Tom, and A.A.**

The command written for this example was….

awk '/916/{print $1, "(916)"}' AwkLab.data.

The command you should write in your file…

awk '/916/{print $1, "(916)"}' Filename.

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.grymoire.com/Unix/Awk.html#uh-53 Pattern search.

Tutoring Assistance.

Question **8.**

Print Sacharissa's **campaign contributions** following her name.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '/Sacharissa/ {print $1, "$" $3, "$" $4, "$" $5}' AwkLab.da
ta
Sacharissa Cripslock $250 $100 $175
aavbox26@Ubuntu1:/mnt/fileshare$
```

The goal of this command was to print the full name of Sacharissa and their campaign contributions. The way I did this was through trial and error and combining my knowledge from previous commands, as well as looking through different sources. We begin with **-F:** which is the field separator, then a **single quote (')** then the pattern/sequence which was Sacharissa, we then write a curly bracket and print, to print the fields that are the **contribution amounts.** The **first field** that is printed is Sacharissa's full name, which is Sacharissa Cripslock.

The **third**, **fourth,** and **fifth fields** that are printed are the contribution amounts**. $3** was for the **250 amount**, **$4** was for the **100 amount**, and finally **$5** was for the **175 amount.** In front of each contribution amount we write a **($) dollar sign** in double quotes **("")** to print the dollar sign in the output for each contribution amount. After writing all the fields we then close the **curly brackets {}** with a **single**

quote (‘) and write the filename we are looking for the data for which is **AwkLab.data.**

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.grymoire.com/Unix/Awk.html#uh-53 Pattern search.

Tutoring Assistance.

Question **9.**

Print **last names** followed by a **comma** and the **phone number.**

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '{split($1, a, " "); print a[length(a)] ", " $2}' AwkLab.da
ta
Vimes, (510) 548-1278
Stibbons, (408) 538-2358
Überwald, (206) 654-6279
Helit, (206) 548-1348
Aching, (206) 548-1278
Cripslock, (916) 343-6410
Dearheart, (406) 298-7744
Baggins, (206) 548-1278
Bombadil, (916) 348-4278
Took, (510) 548-5258
Gamgee, (408) 926-3456
Milne, (916) 440-1763
Saint-Exupery, (123) 978-6432
Took, (345) 978-7684
Took, (453) 978-3534
Took, (123) 978-5754
Took, (123) 978-3574
Took, (345) 978-2677
Took, (563) 978-753
Took, (574) 978-8535
aavbox26@Ubuntu1:/mnt/fileshare$
```

The following command prints all the last names within the file, followed by a comma and prints the phone number associated with the last name. We start this command with **-F:** which is a **field separator**. The field separator tells awk to split the name and phone numbers and keep them in their respective fields**, field 1 for names and field 2 for phone numbers.** We then write a **curly bracket {}** and then the

split function. The split function breaks a string into pieces based on the **delimiter or regular expression** used. The split function within this command takes the first field **($1)** which is the full name and breaks it into an **array** called **(a) based on the spaces between words.**

An **array** is a list of values like numbers or strings called elements. The double quote space double quote **(" ")** is a separator used for splitting the full name. The double quote comma space double quote **(", ")** is a string literal that manually inserts a comma and a space into the output between the name and output.

A string literal is a character that is within double quotes. a[length(a)] Placing length within the square brackets tells awk to print the **last item in the list,** this tells the command to skip the first and middle names in the file and only prints the last names.

The command written in this example was...

awk -F: '{split($1, a, " "); print a [length(a)] ', ' $2}' AwkLab.data.

The command you would write for your file would be...

awk -F: '{split($1, a, " "); print a [length(a)] ', ' $2}' AwkLab.data.

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.grymoire.com/Unix/Awk.html#uh-45 **The Split Function.**

https://www.gnu.org/software/gawk/manual/html_node/Arrays What is an Array?

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.gnu.org/software/gawk/manual/gawk.html#String-Functions Shows examples of split and length commands, as well as arrays.

Question **10.**

Print the **first names** and **last names** of those who **contributed more than 110 dollars** in the last month.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '$5 > 110 {print $1,"$" $5} ' AwkLab.data
Samuel Vimes $175
Ponder Stibbons $201
Susan Sto Helit $175
Tiffany Aching $150
Sacharissa Cripslock $175
Adora Belle Dearheart $275
Tom Bombadil $175
Peregrin Took $135
Samwise Gamgee $200
A.A. Milne $300
Antoine de Saint-Exupery $175
Adalgrim Took $467
Bandobras "Bullroarer" Took $4673
Belladonna Took $175
Eglantine Took $4367
Mirabella Took $175
Ferumbras III Took $3457
Gerontius Took $4562
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

The following command written for this example was…

Awk -F: ‘ $5 > 110 {print $1 , “$” $5}’ AwkLab.data

Breaking down this command awk is the program, **-F:** is the **Field Separator,** we then write a **single quote (‘) $5** dollar sign five for the last field to be searched and the comparison operator **> (greater than)** with the value we want it to be compared to which is **110.** We print an individual's full name and last contribution amount which is the **$5 field**, **“$”** prints the dollar sign character in front of the last contribution amount in the output. We then close the command and write another **single quote (‘),** followed by the filename we are looking for this data in which is AwkLab.data.

Greater than **(>)** Less than **(<).**

Sources:

https://codesignal.com/learn/courses/text-processing-with-bash/lessons/text-processing-with-awk **Printing examples, conditionals, custom line messages for "$".**

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

Question **11.**

Print **last names, phone numbers, and first month** contributions of those who contributed less than **150 dollars** in the first month.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '{sub(/^.* /,"",$1)} $3 < 150 {print $1, $2, "$" $3}' AwkLa
b.data
Aching (206) 548-1278 $15
Took (510) 548-5258 $50
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

The command written for this example was....

awk -F: '{sub (/^.* /,"",$1)} **$3 < 150 {print $1 , $2, "$" $3} AwkLab.data**

The first part of command is searching for the last name. Which is within the first field. **-F:** is the **field separator.** Single quote **(')** is the beginning of the command, we then write a curly bracket, the sub command, parentheses, then we start the search pattern using **Regex** with a **slash (/), ^ caret**, caret signifies the start of a line. We then write a **period (.)**, which means to match any character, the **asterisk (*)** means anything can come after the period **(.) 0 or more times.** After the asterisk it is important to add a single space, the space looks for a space in the text line, we do this because the file is written as first name, space middle name, space last name.

This is how we get to the first name, middle name, last name and stop at the last name because the last name does not have any spaces in them. The **comma (,)** is used to separate sections of the sub command, the two **double quotes ("")** represent our replacement, which is replacing something with nothing, acting as a way to delete the first and middle names from our output. In the output with **field one ($1)** we print the last name, we then print the phone number for the individual which is **field two ($2).**

We then write the **third field ($3),** the third field is the contribution amount that is lower than 150 dollars within the first month. We also write the **"$"** to have it written in the output as well. After this we write our closing **curly bracket {} and a single quote ('),** followed by the filename which is **AwkLab.data.**

Sources:

https://codesignal.com/learn/courses/text-processing-with-bash/lessons/text-processing-with-awk **Printing examples, conditionals, custom line messages for "$".**

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.gnu.org/software/gawk/manual/html_node/String-Functions.html Sub Command.

https://www.gnu.org/software/gawk/manual/html_node/Bracket-Expressions.html Bracket Examples.

Question **12.**

Print the first names and contributions of those who contributed between **10 dollars**

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '{sub(/ .+/,"", $1)} $3 > 10 && $3 < 200 {print $1, "$" $3,
"$" $4,"$" $5}' AwkLab.data
Ponder $155 $90 $201
Tiffany $15 $188 $150
Peregrin $50 $95 $135
A.A. $175 $75 $300
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

and 200 hundred dollars in the first month.

The command written in the example was....

awk -F: '{sub(/ .+/,"", $1)} $3 > 10 && $3 < 200 {print $1, "$" $3, "$" $4, "$" $5}' AwkLab.data

We start this command with awk which is the program we are working with, we then **write -F:** which means Field Separator, then a **single quote (') curly bracket**

{}, then we write the sub command parentheses, then we start the search pattern using **Regex** with a **slash (/), then we add a space in the command because there is a space in front of each last name in the file.**

Then write a **period (.)**, which means to match any character, the **plus (+)** means it looks for one or more characters, it looks for the **character 1 or more times.** After the plus we write another slash **(/)** to close the pattern search which is **([space] .+).** We then write two **double quotes (“”) .** We are targeting the middle name and last name and the whitespace when using the double quotes.

If we write a name like Tom within the quotes it would be inserted into the first field where the last name is. The replacement is what is within the double quotes, the replacement can be anything. For this command the replacement is blank because we are replacing our target with nothing to act as a way to delete the middle name and last names.

We then write the third field **($3)** which is the first contribution, then the greater than symbol **(>) and then 10 because we want to find who contributed between 10 and 200.** We then print the first field which is the individuals first name and the third, fourth and fifth fields are the contribution amounts.

Reminder of symbols:

Greater than **(>)** Less than **(<).**

Sources:

https://codesignal.com/learn/courses/text-processing-with-bash/lessons/text-processing-with-awk **Printing examples, conditionals, custom line messages for “$”.**

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.gnu.org/software/gawk/manual/html_node/String-Functions.html **Sub Command.**

https://www.gnu.org/software/gawk/manual/html_node/Bracket-Expressions.html Bracket Examples.

Question **13.**

<u>Print the **first name, last name, and total contributions** of those who contributed less than 700 dollars over a three month period.</u>

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: ' $3 + $4 + $5 < 700 {print $1." " $3 + $4 + $5}' AwkLab.da
ta
Samuel Vimes 525
Ponder Stibbons 446
Angua von Überwald 360
Susan Sto Helit 525
Tiffany Aching 353
Sacharissa Cripslock 525
Frodo Baggins 405
Tom Bombadil 525
Peregrin Took 280
Samwise Gamgee 618
A.A. Milne 550
Antoine de Saint-Exupery 525
aavbox26@Ubuntu1:/mnt/fileshare$
```

To write this command in awk we would start with the **-F: field separator**, followed by a **single quote (‘)** then we want to add the total contributions together so we write the third field plus the fourth field plus the fifth field **($3 + $4 + $5).** We then want to search for people who have a total contribution that is **less than 700** so we write the **less than symbol (<) then 700.** We then print the full name and the total contribution amount for each individual in the file named **AwkLab.data.**

Reminder of symbols:

Greater than **(>)** Less than **(<).**

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-1 **Printing basic structure.**

https://www.gnu.org/software/gawk/manual/html_node/Arithmetic-Ops.html **Adding fields together in Awk.**

Question **14.**

Print the **first names, and first letter of the last name** and average contribution of those who contributed more than 300 dollars.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '{split($1, name, " "); avg=($3+$4+$5)/3; if(avg>300) print
(name[1],substr(name[length(name)],1,1), "$" avg)}' AwkLab.data
Adora D $341.667
Adalgrim T $1746.67
Bandobras T $3931.33
Eglantine T $1771.67
Mirabella T $507
Ferumbras T $1269
Gerontius T $1925
aavbox26@Ubuntu1:/mnt/fileshare$
```

The full command written for this example was

awk -F: ‘{split($1, name, “ “); avg=($3+$4+$5)/3; if(avg>300) print (name[1], substr(name[length(name)], 1, 1), “$” avg)}’ AwkLab.data

We start with the **-F:** which is a field separator, we then write a **single quote (‘)**, then a **curly bracket {},** we then write the split command, parentheses **(),** then field one **$1** which is the full name of a person within the file. We then write our array which can be named anything, for this command to break up the name I wrote the array as the word **“name”.**

We then write a double quote space double quote **(" "),** this is what split is looking for, what do we want to split the name with. You want to separate only the first field with spaces. Then a semi colon **(;),** then we create a variable called **avg** or **(Average)**, we then write a equals sign **(=)**, then we add up the contribution amounts which are in fields three, four, and five **($3,$4,$5).**

Then we write a **forward slash (/)** and a three **(3)** to divide the contribution amounts by three, we want to divide it by three because we want to find the average because we want to divide by the same number we add up to. We then write a semi colon **(;),** then an if statement because we want to know if the average is greater than three hundred **(300).** We then print the first element which is the **first name** in the string of field one, which is stored in the **“name”** array.

We then write a **substring** command, parentheses name array, then a **square bracket [],** then we write length because we want to know the length of

the last element. Then we write the name array which we then write **1 , 1** which tells the **substring** command which character **1** is the start of the last name and then we want to end at character **1** which would only get us the first letter of the last name. We could write **1, 3** and it would get us the first three letters of the last name.

We then write dollar sign in double quotes **"$"** to print the dollar sign in each contribution amount output. We then print the average and close the command with parentheses **()** and then a curly bracket **{}**. We write a single quote **(')** followed by the file name AwkLab.data.

Reminder of symbols:

Greater than **(>)** Less than **(<).**

Steps to remember Split:

split**(1.** string, **2**. Array **3.**[,fieldsep]).

split (**1.** the string you want to work with, **2.** giving a name to the array where you want to store the string, setting a field separator for that string, **3.** spacing with " ", separator like semi colon).

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.gnu.org/software/gawk/manual/html_node/Arithmetic-Ops.html Adding fields together in Awk.

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.gnu.org/software/gawk/manual/html_node/String-Functions.html **What Split does and examples.**

https://codesignal.com/learn/courses/text-processing-with-bash/lessons/text-processing-with-awk **Printing examples, conditionals, custom line messages for "$".**

Tutoring Assistance.

Question **15.**

<u>Print the **last name and the area code** for those not in the **916 area code.**</u>

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '!/916/ {sub(/^.* /, "", $1); sub(/ .*$/, "", $2); print $1
, $2}' AwkLab.data
Vimes (510)
Stibbons (408)
Überwald (206)
Helit (206)
Aching (206)
Dearheart (406)
Baggins (206)
Took (510)
Gamgee (408)
Saint-Exupery (123)
Took (345)
Took (453)
Took (123)
Took (123)
Took (345)
Took (563)
Took (574)
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

The command written for this example was….

awk -F: ‘ !/916/ {sub(/^.* /,””, $1); sub(/ .*$/, ””, $2); **print $1, $2}’ AwkLab.data**

Invert matching in awk symbol: **(!) exclamation mark.**

The first part of command is searching for the last name. Which is within the first field. **-F:** is the **field separator.** Single quote **(‘)** is the beginning of the command, we write an **exclamation mark (!) the exclamation mark means …**then the search/pattern sequence we are **excluding** which is **916**, we then write a curly bracket, the <u>sub</u> command, parentheses, then we start the search pattern using **Regex** with a **slash (/), ^ caret**, caret signifies the start of a line.

We then write a **period (.)**, which means to match any character, the **asterisk (*)** means anything can come after the period **(.) 0 or more times.** After the asterisk it is important to add a single space, the space looks for a space in the text line, we do this because the file is written as first name, space middle name, space last name.

This is how we get to the first name, middle name, last name and stop at the last name because the last name does not have any spaces in them.The **comma (,)** is used to separate sections of the sub command, the two **double quotes (””)** represent our replacement, which is replacing something with nothing, acting as a way to delete the first and middle names from our output. In the output with **field one ($1)** we print the last name.

For the second sub command, we write a semi colon to separate the sub commands, then a space followed by sub to start the command for the second field. Inside the parentheses we start with a space, we then write a **period (.),** then an **asterisk (*)** to match every character after the area code. We then write a **dollar sign ($)** at the end, which means deleting everything from the space to the end of the field.

We then write two double quotes **(””),** the double quotes represent our replacement which is replacing something with nothing, it acts as a way to delete the rest of the phone number from our output so only the **area codes that are not 916 remain.** Finally, when we print the first field **$1** we get the last name followed by the second field **$2** which is the area code for everyone not in the 916 area code.

Steps to remember for the sub command and how it works:

Substitution (regular expression, what we want to replace (like a name), [, what the target is (like a name change)]).

Another way to think about the sub command:

sub(what am I looking for, what do I want to change it to, which field am I changing).

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.gnu.org/software/gawk/manual/html_node/String-Functions.html **Sub Command.**

https://www.gnu.org/software/gawk/manual/gawk.html#Precedence What is an exclamation mark in Awk. (!).

Question **16.**

<u>Print each record</u> <u>preceded by the number of the record.</u>

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '{print NR, $0}' AwkLab.data
1 Samuel Vimes:(510) 548-1278:250:100:175
2 Ponder Stibbons:(408) 538-2358:155:90:201
3 Angua von Überwald:(206) 654-6279:250:60:50
4 Susan Sto Helit:(206) 548-1348:250:100:175
5 Tiffany Aching:(206) 548-1278:15:188:150
6 Sacharissa Cripslock:(916) 343-6410:250:100:175
7 Adora Belle Dearheart:(406) 298-7744:450:300:275
8 Frodo Baggins:(206) 548-1278:250:80:75
9 Tom Bombadil:(916) 348-4278:250:100:175
10 Peregrin Took:(510) 548-5258:50:95:135
11 Samwise Gamgee:(408) 926-3456:250:168:200
12 A.A. Milne:(916) 440-1763:175:75:300
13 Antoine de Saint-Exupery:(123) 978-6432:250:100:175
14 Adalgrim Took:(345) 978-7684:4673:100:467
15 Bandobras "Bullroarer" Took:(453) 978-3534:6753:368:4673
16 Belladonna Took:(123) 978-5754:356:247:175
17 Eglantine Took:(123) 978-3574:473:475:4367
18 Mirabella Took:(345) 978-2677:783:563:175
19 Ferumbras III Took:(563) 978-753:250:100:3457
20 Gerontius Took:(574) 978-8535:535:678:4562
aavbox26@Ubuntu1:/mnt/fileshare$
```

The command written for this example was….

awk -F: ‘{print NR, $0}’ AwkLab.data

We start the command with awk the program we are working with, we then write the **-F:** which is a <u>Field Separator.</u> We then write a **single quote (‘)** then a **curly bracket {},** then <u>print,</u> then we write **“NR”** which means <u>number of records</u> and we want to print it.**The comma (,)** is written for clarity, to see each number record for each line in the file named AwkLab.data. We then write dollar sign zero **$0**, to **print every line** in the file as it is originally written.

Important things to remember in command written:

NR stands for **number of records.**

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.geeksforgeeks.org/linux-unix/awk-command-unixlinux-examples/ Number of records command examples.

Question **17.**

Print the name and **total contribution** of **each person**

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '{print $1, "total contribution:", "$" $3+$4+$5}' AwkLab.da
ta
Samuel Vimes total contribution: $525
Ponder Stibbons total contribution: $446
Angua von Überwald total contribution: $360
Susan Sto Helit total contribution: $525
Tiffany Aching total contribution: $353
Sacharissa Cripslock total contribution: $525
Adora Belle Dearheart total contribution: $1025
Frodo Baggins total contribution: $405
Tom Bombadil total contribution: $525
Peregrin Took total contribution: $280
Samwise Gamgee total contribution: $618
A.A. Milne total contribution: $550
Antoine de Saint-Exupery total contribution: $525
Adalgrim Took total contribution: $5240
Bandobras "Bullroarer" Took total contribution: $11794
Belladonna Took total contribution: $778
Eglantine Took total contribution: $5315
Mirabella Took total contribution: $1521
Ferumbras III Took total contribution: $3807
Gerontius Took total contribution: $5775
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

The command written for this example was….

awk -F: ‘ {print $1, “total contribution $ $3+$4+$5”}’ AwkLab.data

The command you should write for your file….

awk -F: ‘ {print $1, “total contribution $ $3+$4+$5”}’ Filename

We start this command with awk, followed by **-F:** which is a field separator. **-F:** separates the name, phone numbers, and monthly contributions. We then write a **single quote (‘)**, then a **curly bracket {}** the print statement, then we want to print the first field which is the full name. We then write a **double quote (“),** inside the double quotes is a string literal that says **“total contribution followed by a colon (:),** we then end this string with another **double quote (“).**

A string literal is like a message within a code line or command. We then write a **dollar sign "$"** within **double quotes (")** to print the dollar sign **($)** in front of each contribution for each individual. To get the total contribution we add each individual contribution together.

Each contribution is in its own field, contribution **1** is in field **3**, contribution **2** is in field **4,** and finally contribution **3** is in field **5.** After adding each contribution together we end the command by closing it in a **curly bracket {},** we then write a **single quote ('),** then we write the filename which is AwkLab.data.

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.gnu.org/software/gawk/manual/html_node/Arithmetic-Ops.html Adding fields together in Awk.

Question **18.**

Add 10 dollars to **Tiffany Aching's** first contribution and **print her full name and first contribution.**

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '/Tiffany Aching/{print $1,$3+10}' AwkLab.data
Tiffany Aching 25
aavbox26@Ubuntu1:/mnt/fileshare$
```

The command written for this example was.....

awk -F: ' /Tiffany Aching/ {print $1,$3+10}' AwkLab.data

We start writing this command by writing the awk program, followed by the **-F:** field separator, then we write a **single quote (')**. We then write a **forward slash (/),** we then write the pattern/sequence we are looking for which is the name in the file **"Tiffany Aching".** We close this pattern search with another **forward slash (/),** we

then write a **curly bracket {},** then the print statement, we then search for field one **($1)** because we want to print **Tiffany Aching's** full name.

We then write field three **($3)** to look for the first contribution and then we write a **plus sign (+)** and then the **number ten (10),** the reason we write this is because we want to **add 10 dollars** to the total contribution amount that Tiffany Aching has originally. **Tiffany Aching** had **$15** for the **first contribution** that was in **field three ($3).** If you **add ten (10)** to the **original contribution amount** it is **$25.** If we **cat AwkLab.data** you can see her original contribution amount is **15** and we **add ten it is 25.**

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.grymoire.com/Unix/Awk.html#uh-53 Pattern Searching.

https://www.grymoire.com/Unix/Awk.html#uh-1 Printing basic structure.

https://www.gnu.org/software/gawk/manual/html_node/Arithmetic-Ops.html Adding fields together in Awk.

Question **19.**

Changing Samwise Gamgee's **name** to Sean Astin.

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -F: '/Samwise Gamgee/{sub(/Samwise Gamgee/, "Sean Astin"); prin
t $0}' AwkLab.data
Sean Astin:(408) 926-3456:250:168:200
aavbox26@Ubuntu1:/mnt/fileshare$ _
```

The full command written for this example was....

awk -F: '/Samwise Gamgee/ {sub(/Samwise Gamgee/, "Sean Astin"); print $0}' AwkLab.data

We start writing this command by writing awk then the **-F: field separator,** we then write a **single quote (‘),** we then write a **forward slash (/)** to begin the pattern/sequence search which is Samwise Gamgee, we close the search with another **forward slash (/).** We then write a **curly bracket {}** then we start writing out the sub command, then parentheses **().**

Inside the parentheses we write the search pattern which is the full name Samwise Gamgee, we then write a **comma (,)** we write the comma to separate the name we want to find with the name we want to replace it with, which would be **“Sean Astin”** in double quotes. Adding the double quotes ensures that the name Samwise Gamgee is substituted with the name **Sean Astin** which is a string literal, like a message to change the name to the substituted name.

We then write a **semi colon (;)** to separate the substitution from the next step, which is to write a print statement and then **dollar sign zero ($0)** because we want to get the same original line of Samwise Gamgee with the successful substituted name which is Sean Astin. Finally we close the command with a **curly bracket {},** then a single quote and then write the filename, which in this case would be AwkLab.data.

Steps to remember for the sub command and how it works:

Substitution (regular expression, what we want to replace (like a name), [, what the target is (like a name change)]).

Another way to think about the sub command:

sub(what am I looking for, what do I want to change it to, which field am I changing).

Sources:

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

https://www.gnu.org/software/gawk/manual/html_node/String-Functions.html **Sub Command.**

https://www.grymoire.com/Unix/Awk.html#uh-53 Pattern Searching.

Question **20.**

1. Writing an **awk script** that prints the **first name** of all the people with the last name **"Took"** followed by their campaign contributions.

   ' /Took/ {split($1, name, " "); amount=($3+$4+$5); print (name [1], "total amount", "$", amount)'.

2. Printing the **full names** and contributions of anyone who contributed between **10 dollars and 200 dollars** in the last contribution.

   If ($5 >= 10) greater than

   If ($5 <= 200) less than

   Print $1 -full name

   $5 is contribution **3** and the last contribution amount in the file AwkLab.data.

Full command: if($5>=10 && $5<=200) print ($1, "$"$5)}

Reminder of symbols:

Greater than **(>)** Less than **(<).**

3. Print the full names and average contribution of those who contributed less than **300 dollars** on average.

avg = ($3+$4+$5)/3.

If (avg < 300) **if the average is less than 300.**

Less than **(<).**

To print the full name we want to print field one ($1), then the avg.

Full command: {avg=$3+$4+$5)/3 -first line

Second line- if(avg<300) print($1, "average", "$", avg)}} .

The Shebang line command is **"which awk".**

Which awk is for me, when writing the command : **/usr/bin/awk**

Full Example Command**:**

#!/usr/bin/awk -f

Write command line.

Begin {FS= ":";

/Took/ {split($1, name, " "); amount=($3+$4+$5); print (name [1], "total amount", "$", amount)} Part I- Script for the first name of people with the last name "Took" and their contribution amounts.

if($5>=10 && $5<=200) print ($1, "$"$5)} Part II- Second line of script that looks for the full name and contributions of those who contributed between 10 and 200 dollars.

{avg=$3+$4+$5)/3 -first line

Second line- **if(avg<300) print($1, "average", "$", avg)}}**

Part III- Final parts of script that print the full name and contributions less than 300.

Screenshot of Awk Script:

```
  GNU nano 6.2                                   awkscript.awk
#!/usr/bin/awk/-f

BEGIN { FS= ":"}

/Took/ {amount=($3+$4+$5)

 split($1,name, " ")

 print(name[1],"total amount","$",amount)}

{if($5>=10 && $5<=200) print($1,"$"$5)}

{avg=($3+$4+$5)/3

if(avg<300) {print ($1,"average","$", avg)}}
```

```
aavbox26@Ubuntu1:/mnt/fileshare$ awk -f awkscript.awk AwkLab.data
Samuel Vimes $175
Samuel Vimes average $ 175
Ponder Stibbons average $ 148.667
Angua von Überwald $50
Angua von Überwald average $ 120
Susan Sto Helit $175
Susan Sto Helit average $ 175
Tiffany Aching $150
Tiffany Aching average $ 117.667
Sacharissa Cripslock $175
Sacharissa Cripslock average $ 175
Frodo Baggins $75
Frodo Baggins average $ 135
Tom Bombadil $175
Tom Bombadil average $ 175
Peregrin total amount $ 280
Peregrin Took $135
Peregrin Took average $ 93.3333
Samwise Gamgee $200
Samwise Gamgee average $ 206
A.A. Milne average $ 183.333
Antoine de Saint-Exupery $175
Antoine de Saint-Exupery average $ 175
Adalgrim total amount $ 5240
Bandobras total amount $ 11794
Belladonna total amount $ 778
Belladonna Took $175
Belladonna Took average $ 259.333
Eglantine total amount $ 5315
Mirabella total amount $ 1521
Mirabella Took $175
Ferumbras total amount $ 3807
Gerontius total amount $ 5775
aavbox26@Ubuntu1:/mnt/fileshare$
```

Output of awk script.

FS is a variable. It stands for **Field Separator. -F:** is also another **field separator in awk.**

Written in terminal (first step) :

sudo chmod awkscript.awk.

How to write script:

Write nano file.awk or script.awk etc.

How to run awk script:

awk -f awkscript.awk AwkLab.data.

Sources:

https://www.youtube.com/watch?v=4HpWO_RAMq4&t=152s Used **example** from video to understand what a script is for the awk programming language.

https://www.grymoire.com/Unix/Awk.html#uh-15 **Field Separators.**

Also Tutoring assistance.